

Universidad Nacional de San Antonio Abad del Cusco

Departamento Académico de Ingeniería Informática y de Sistemas

DESARROLLO DE APLICACIONES MÓVILES

Práctica N° 1

INTRODUCCIÓN A FLUTTER EN ANDROID STUDIO

1. OBJETIVO.

- Conocer el entorno de desarrollo Android Studio y su integración con Flutter.
- Instalar y configurar el SDK de Flutter y las dependencias necesarias.
- Comprender la estructura de un proyecto Flutter y el rol del widget tree.
- Crear y ejecutar la primera aplicación Flutter en un emulador Android.
- Explorar los widgets fundamentales: Text, Container, Row, Column e Image.

2. BASE TEÓRICA COMPLEMENTARIA

2.1. ¿Qué es Flutter?

Flutter es un framework de código abierto creado por Google para construir aplicaciones nativas compiladas para móvil (Android e iOS), web y escritorio, a partir de una única base de código escrita en el lenguaje Dart. A diferencia de otros frameworks híbridos, Flutter no usa componentes nativos del SO: renderiza su propia interfaz mediante el motor gráfico Skia/Impeller, logrando 60 o 120 fps con apariencia consistente en todas las plataformas.

Característica	Descripción
Lenguaje	Dart (compilado AOT a código nativo ARM)
Renderizado	Motor propio Skia / Impeller (no usa WebView ni puentes nativos)
Plataformas	Android, iOS, Web, Windows, macOS, Linux
Hot Reload	Recarga de UI en milisegundos sin perder el estado
Arquitectura UI	Todo es un Widget (composición declarativa)

2.2. El Lenguaje Dart

Dart es un lenguaje orientado a objetos, fuertemente tipado y con null-safety. Su sintaxis es similar a C# y Java, lo que facilita la curva de aprendizaje para desarrolladores con experiencia en lenguajes de la familia C.

```
// Hola Mundo en Dart
```

```

void main() {
  String nombre = 'Flutter';
  int version = 3;
  print('Hola desde \${nombre} v\${version}');
}

// Funciones con tipos y valores opcionales
int sumar(int a, int b) => a + b;

// Null-safety: el signo ? indica que puede ser nulo
String? obtenerNombre() => null;

```

2.3. Concepto de Widget

En Flutter, todo es un Widget: botones, textos, columnas, márgenes, animaciones e incluso la propia aplicación. Los widgets se organizan en un árbol **jerárquico llamado Widget Tree**. Flutter reconstruye eficientemente este árbol cada vez que el estado cambia, actualizando solo las partes necesarias de la pantalla.

Existen dos categorías principales de widgets:

- **StatelessWidget**: widget inmutable cuya apariencia no cambia en el tiempo. Ideal para contenido estático.
- **StatefulWidget**: widget que mantiene un estado mutable y puede actualizarse dinámicamente usando `setState()`.

```

// StatelessWidget - no tiene estado interno
class Saludo extends StatelessWidget {
  final String nombre;
  const Saludo({required this.nombre, super.key});

  @override
  Widget build(BuildContext context) {
    return Text('Hola, \${nombre}!');
  }
}

// StatefulWidget - tiene estado que puede cambiar
class Contador extends StatefulWidget {
  const Contador({super.key});
  @override
  State<Contador> createState() => _ContadorState();
}

class _ContadorState extends State<Contador> {
  int _cuenta = 0;

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text('Cuenta: \$_cuenta'),
      ElevatedButton(
        onPressed: () => setState(() => _cuenta++),
        child: const Text('Incrementar'),
      ),
    ],

```

```
    });  
  }  
}
```

2.4. Widgets de Layout Fundamentales

Flutter provee widgets de layout para organizar otros widgets en la pantalla:

Widget	Función principal
Container	Caja con padding, margin, color, bordes y transformaciones
Row	Dispone hijos en sentido horizontal (eje X)
Column	Dispone hijos en sentido vertical (eje Y)
Stack	Superpone widgets uno sobre otro (como capas)
Expanded	Ocupa el espacio restante disponible dentro de Row/Column
ListView	Lista desplazable de widgets de longitud variable
GridView	Cuadrícula bidimensional de widgets
Scaffold	Estructura base de pantalla (AppBar, body, FAB, Drawer)

2.5. pubspec.yaml — Gestión de Dependencias

El archivo pubspec.yaml es el manifiesto del proyecto Flutter. Define el nombre, la versión, las dependencias de paquetes (pub.dev), los assets (imágenes, fuentes) y la versión mínima de SDK.

```
name: mi_primera_app  
description: Práctica introductoria de Flutter.  
version: 1.0.0+1  
  
environment:  
  sdk: '>=3.0.0 <4.0.0'  
  
dependencies:  
  flutter:  
    sdk: flutter  
  cupertino_icons: ^1.0.6  
  http: ^1.2.0          # Paquete para peticiones HTTP  
  
flutter:  
  uses-material-design: true  
  assets:  
    - assets/images/    # Carpeta de imágenes locales
```

3. INSTALACIÓN Y CONFIGURACIÓN DEL ENTORNO

3.1. Requisitos del Sistema

Componente	Versión mínima recomendada
Sistema Operativo	Windows 10/11 (64-bit), macOS 12+, Ubuntu 20.04+
Espacio en disco	10 GB libres (SDK + Android Studio + emulador)
RAM	8 GB mínimo, 16 GB recomendado
Android Studio	Flamingo 2022.2.1 o superior
Flutter SDK	3.x (canal stable)
Dart SDK	Incluido con Flutter 3.x
JDK	Java 17 (incluido en Android Studio)

3.2. Instalación de Flutter SDK

Siga los pasos en orden para instalar Flutter correctamente:

1. Descargar el SDK estable desde <https://flutter.dev/docs/get-started/install>
2. Descomprimir el archivo en una ruta sin espacios ni caracteres especiales (ej. C:\flutter en Windows).
3. Agregar la carpeta flutter\bin al PATH del sistema.
4. Verificar la instalación ejecutando en terminal:

```
flutter doctor

# Salida esperada (cada item debe mostrar un check verde):
# [✓] Flutter (Channel stable, 3.x.x)
# [✓] Android toolchain
# [✓] Android Studio (version Flamingo+)
# [✓] Connected device
```

3.3. Configuración de Android Studio

1. Abrir Android Studio → File → Settings → Plugins.
2. Buscar 'Flutter' en el Marketplace e instalar el plugin (instala Dart automáticamente).
3. Reiniciar Android Studio.
4. Ir a File → Settings → Languages & Frameworks → Flutter.
5. Especificar la ruta del Flutter SDK (carpeta descomprimida en el paso anterior).
6. Crear un emulador Android: Tools → Device Manager → Create Device.

 **Consejo:** Seleccionar un dispositivo Pixel con API level 33 (Android 13) para mayor compatibilidad con las APIs modernas de Flutter.

4. DESARROLLO DE LA PRÁCTICA

A continuación se presenta el desarrollo paso a paso de tres ejercicios progresivos. Cada ejercicio agrega complejidad sobre el anterior, consolidando los conceptos aprendidos.

4.1. Ejercicio 1: Crear y Ejecutar el Primer Proyecto Flutter

Paso 1 — Crear el proyecto

1. En Android Studio: File → New → New Flutter Project.
2. Seleccionar Flutter Application y hacer clic en Next.
3. Nombre del proyecto: mi_primera_app (sin espacios ni mayúsculas).
4. Elegir la ruta del proyecto y el Flutter SDK. Hacer clic en Finish.

Paso 2 — Estructura del proyecto generado

Archivo / Carpeta	Descripción
lib/main.dart	Punto de entrada de la aplicación (función main)
android/	Código nativo Android y configuración Gradle
ios/	Código nativo iOS y configuración Xcode
pubspec.yaml	Dependencias, assets y metadatos del proyecto
test/	Tests unitarios y de widget
.dart_tool/	Caché interna de Dart (no editar)

Paso 3 — Código base explicado

```
// lib/main.dart
import 'package:flutter/material.dart';

// Punto de entrada: ejecuta la aplicación
void main() {
  runApp(const MiApp());
}

// Widget raíz - configura MaterialApp (tema, rutas, título)
class MiApp extends StatelessWidget {
  const MiApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Mi Primera App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
        useMaterial3: true,
      ),
      home: const PantallaInicio(),
    );
  }
}
```

```

    }
}

// Pantalla principal con Scaffold
class PantallaInicio extends StatelessWidget {
  const PantallaInicio({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Práctica Flutter'),
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
      ),
      body: const Center(
        child: Text(
          '¡Hola, Flutter!',
          style: TextStyle(fontSize: 28, fontWeight: FontWeight.bold),
        ),
      ),
    );
  }
}

```

Paso 4 — Ejecutar la aplicación

1. Iniciar el emulador desde Tools → Device Manager → Play.
2. Presionar el botón Run (triángulo verde) o usar Shift+F10.
3. Para recargar cambios sin reiniciar: guardar el archivo (Ctrl+S) → Hot Reload automático.

⚠ Nota: El primer build puede tardar 3-5 minutos mientras Gradle descarga las dependencias. Los builds posteriores son mucho más rápidos.

4.2. Ejercicio 2: Tarjeta de Perfil con Layout Widgets

Se construye una pantalla con la tarjeta de perfil de un usuario, utilizando Container, Row, Column, CircleAvatar y Card. Reemplazar el contenido de PantallaInicio con el siguiente código:

```

class PantallaInicio extends StatelessWidget {
  const PantallaInicio({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Tarjeta de Perfil'),
        backgroundColor: Colors.blue.shade800,
        foregroundColor: Colors.white,
      ),
      backgroundColor: Colors.grey.shade100,
      body: Center(

```

```

child: Card(
  elevation: 8,
  shape: RoundedRectangleBorder(
    borderRadius: BorderRadius.circular(20),
  ),
  child: Padding(
    padding: const EdgeInsets.all(24.0),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        // Avatar circular
        CircleAvatar(
          radius: 60,
          backgroundColor: Colors.blue.shade200,
          child: const Icon(Icons.person, size: 70, color: Colors.white),
        ),
        const SizedBox(height: 16),
        // Nombre
        const Text('Ada Lovelace',
          style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold)),
        const SizedBox(height: 4),
        Text('Ingeniería Informática',
          style: TextStyle(fontSize: 16, color: Colors.grey.shade600)),
        const Divider(height: 32),
        // Fila de estadísticas
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            _Estadistica(valor: '120', etiqueta: 'Proyectos'),
            _Estadistica(valor: '4.8', etiqueta: 'Rating'),
            _Estadistica(valor: '5+', etiqueta: 'Años'),
          ],
        ),
        const SizedBox(height: 20),
        // Botón de acción
        ElevatedButton.icon(
          onPressed: () {},
          icon: const Icon(Icons.message),
          label: const Text('Enviar Mensaje'),
          style: ElevatedButton.styleFrom(
            backgroundColor: Colors.blue.shade700,
            foregroundColor: Colors.white,
            padding: const EdgeInsets.symmetric(horizontal: 32, vertical:
12),
            shape: RoundedRectangleBorder(
              borderRadius: BorderRadius.circular(30)),
          ),
        ),
      ],
    ),
  ),
);
}
}

// Widget auxiliar para las estadísticas

```

```

class _Estadistica extends StatelessWidget {
  final String valor;
  final String etiqueta;
  const _Estadistica({required this.valor, required this.etiqueta});

  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text(valor, style: const TextStyle(
          fontSize: 22, fontWeight: FontWeight.bold, color: Colors.blue)),
        Text(etiqueta, style: TextStyle(color: Colors.grey.shade600)),
      ],
    );
  }
}

```

4.3. Ejercicio 3: Contador Interactivo con StatefulWidget

Este ejercicio implementa un contador con tres botones: incrementar, decrementar y reiniciar. Demuestra el uso de `setState()` para actualizar la UI en respuesta a eventos de usuario.

```

class PantallaInicio extends StatefulWidget {
  const PantallaInicio({super.key});
  @override
  State<PantallaInicio> createState() => _PantallaInicioState();
}

class _PantallaInicioState extends State<PantallaInicio> {
  int _contador = 0;

  void _incrementar() => setState(() => _contador++);
  void _decrementar() => setState(() { if (_contador > 0) _contador--; });
  void _reiniciar() => setState(() => _contador = 0);

  Color get _colorContador {
    if (_contador == 0) return Colors.grey;
    if (_contador < 5) return Colors.blue;
    return Colors.green;
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Contador Interactivo'),
        backgroundColor: Colors.blue.shade800,
        foregroundColor: Colors.white,
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text('Valor actual:', style: TextStyle(fontSize: 20)),

```



```

const SizedBox(height: 12),
// El color cambia según el valor
AnimatedDefaultTextStyle(
  duration: const Duration(milliseconds: 300),
  style: TextStyle(
    fontSize: 80, fontWeight: FontWeight.bold, color: _colorContador),
  child: Text('\$_contador'),
),
const SizedBox(height: 40),
// Botones de control
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    FloatingActionButton(
      heroTag: 'dec',
      onPressed: _decrementar,
      backgroundColor: Colors.red.shade400,
      child: const Icon(Icons.remove, color: Colors.white),
    ),
    const SizedBox(width: 20),
    FloatingActionButton.extended(
      heroTag: 'rst',
      onPressed: _reiniciar,
      backgroundColor: Colors.grey.shade600,
      label: const Text('Reset', style: TextStyle(color:
Colors.white)),
      icon: const Icon(Icons.refresh, color: Colors.white),
    ),
    const SizedBox(width: 20),
    FloatingActionButton(
      heroTag: 'inc',
      onPressed: _incrementar,
      backgroundColor: Colors.green.shade500,
      child: const Icon(Icons.add, color: Colors.white),
    ),
  ],
),
),
),
),
);
}
}

```

5. CUESTIONARIO

- ¿Cuál es la diferencia fundamental entre un StatelessWidget y un StatefulWidget? Proporcione un ejemplo de uso de cada uno.
- ¿Por qué Flutter no usa componentes nativos del sistema operativo para renderizar su interfaz?
- Explique qué hace el comando flutter doctor y para qué sirve cada sección de su resultado.
- ¿Qué sucede internamente cuando se llama setState()? Describa el proceso de reconstrucción del widget tree.
- ¿Cuál es la diferencia entre Hot Reload y Hot Restart en Flutter?

6. Ejercicio Propuesto.

Desarrollar una aplicación Flutter en Android Studio que implemente una calculadora básica con la siguiente funcionalidad:

- Interfaz con un display numérico que muestre el número actual y el resultado de la operación.
- Botones para los dígitos del 0 al 9, punto decimal, y las cuatro operaciones aritméticas básicas (+, -, ×, ÷).
- Botón C (clear) que borre el display, y botón = que calcule y muestre el resultado.
- El color del display debe cambiar según si el resultado es positivo (azul), negativo (rojo) o cero (gris).
- Implementar usando StatefulWidget, organizando los botones con GridView o una combinación de Row y Column.

 **Entrega:** Subir al aula virtual con capturas de pantalla del emulador y el **link del repositorio**.

BIBLIOGRAFÍA

- Google LLC; Flutter Documentation — Get Started; <https://docs.flutter.dev/get-started/install>
- Google LLC; Dart Language Tour; <https://dart.dev/language>
- Vanda Nahavandipour; Flutter & Dart Cookbook; O'Reilly Media; 2023.
- Eric Windmill; Flutter in Action; Manning Publications; 2nd Edition; 2023.
- Marco Napoli; Beginning Flutter: A Hands-On Guide to App Development; Wiley; 2020.